Lecture 4: ML-DSA (Dilithium) and FN-DSA (Falcon)

Signing With Lattices: Fiat–Shamir-With-Aborts and Trapdoor Sampling

Dr. Faisal Aslam

Lecture Notes — Session 3b (approx. 2 hours)

Purpose of this lecture. Lecture 3 built an encryption scheme out of MLWE. Today we build *signature* schemes — a different goal (proving you hold a secret, not hiding a message) that leads to genuinely different designs. We cover **ML-DSA (Dilithium)**, which reuses MLWE directly inside a technique called Fiat–Shamir-with-Aborts, and **FN-DSA (Falcon)**, which uses a different lattice (NTRU) and a different technique (GPV trapdoor sampling) entirely. Both sections end by naming the exact step in each algorithm that Session 5's physical attacks target — that mapping is the main deliverable of this lecture.

Contents

1 What a Signature Scheme Needs to Do

Encryption (Lecture 3) hides information; a signature does the opposite — it *proves* something publicly, without hiding anything. A signature scheme has three algorithms: KeyGen (produces a public/secret key pair), $\mathrm{Sign}(sk, \text{message})$ (produces a signature), and $\mathrm{Verify}(pk, \text{message}, \text{signature})$ (accepts or rejects). The security goal is **unforgeability**: without sk, no one should be able to produce a signature that Verify accepts, even after seeing many genuine signatures on messages of their choosing.

Remark: The Core Design Tension

A signature must depend on the secret key (otherwise anyone could forge it) — but it must not depend on the secret key in a way that *leaks* it, even after an attacker collects thousands of signatures. Both schemes in this lecture are, at heart, different engineering answers to that one tension. Keep this framing in mind; it's the thread connecting today's material straight through to Session 5.

2 A Primer on Fiat–Shamir (Needed Before Dilithium Makes Sense)

Definition: Fiat–Shamir Transform, Informally

Start with a three-move *interactive* protocol between a Prover (who knows a secret) and a Verifier: (1) Prover sends a **commitment** w (built from fresh randomness, independent of the secret); (2) Verifier sends a random **challenge** c; (3) Prover computes a **response** z that combines w's randomness with the secret and c, and the Verifier checks a consistency equation relating w, c, z, and the public key. The Fiat–Shamir transform removes the Verifier: the Prover computes c themself, as a hash of w (and the message being signed), $c = H(w, \text{message})$. Because H behaves unpredictably, the Prover still can't predict c before committing to w — so the security argument survives, but now there's no interaction: the whole transcript (w, c, z) can be published as a non-interactive signature.

Remark: Why This Detour Was Necessary

Every part of Dilithium's Sign algorithm below — the commitment w, the challenge c, the response z — is exactly this three-move shape, with the interaction removed via hashing. Without seeing the general pattern first, Dilithium's algorithm looks like an arbitrary pile of steps; with it, each line has an obvious job.

3 ML-DSA (Dilithium): Parameters

Definition: ML-DSA Parameter Sets

All parameter sets share the familiar ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ with $n = 256$, this time with $q = 8{,}380{,}417$ (a different prime from Kyber's — still chosen for NTT-friendliness). The secret is a pair of small vectors $(\vec{s}_1, \vec{s}_2) \in R_q^\ell \times R_q^k$; the public key is built from an MLWE-style sample using them.

Parameter set	(k, ℓ)	Signature size	Public-key size
ML-DSA-44	(4,4)	2,420 bytes	1,312 bytes
ML-DSA-65	(6,5)	3,309 bytes	1,952 bytes
ML-DSA-87	(8,7)	4,627 bytes	2,592 bytes

4 ML-DSA: KeyGen, Sign, Verify

Algorithm 1 ML-DSA.KeyGen

1: Generate $A \in R_q^{k \times \ell}$ uniformly at random (again, from a short public seed)
2: Sample small $\vec{s}_1 \in R_q^\ell$, $\vec{s}_2 \in R_q^k$ (coefficients bounded by a small η)
3: Compute $\vec{t} = A\vec{s}_1 + \vec{s}_2 \in R_q^k$
4: **return** $pk = (A, \vec{t})$, $sk = (\vec{s}_1, \vec{s}_2)$

Remark: Yes, This Is an MLWE Sample Too

Line 3 is identical in shape to Kyber's KeyGen (Lecture 3). The two schemes start from the *same* hard problem; what differs entirely is what you do with the key afterward.

Algorithm 2 ML-DSA.Sign$(sk = (\vec{s}_1, \vec{s}_2),\ \text{msg})$ — Simplified

1: Sample a masking vector $\vec{y} \in R_q^\ell$ with somewhat larger small coefficients than $\vec{s}_1, \vec{s}_2$ (derived deterministically from sk and msg via a PRF, not from fresh randomness — see remark below)
2: Compute the commitment $\vec{w} = A\vec{y} \in R_q^k$
3: Compute the challenge $c = H(\vec{w}, \text{msg})$ — a specially-shaped "sparse" small polynomial
4: Compute the response $\vec{z} = \vec{y} + c\vec{s}_1$
5: **Rejection check:** if any coefficient of $\vec{z}$ (or of a related quantity built from $\vec{w} - c\vec{s}_2$) is too large in absolute value, **discard everything and go back to step 1** with fresh $\vec{y}$
6: Once the check passes, **return** signature $(\vec{z}, c)$ (in the real scheme, a compact *hint* $\vec{h}$ is also included, letting the verifier reconstruct $\vec{w}$'s high-order bits exactly — omitted here for simplicity)

Algorithm 3 ML-DSA.Verify$(pk = (A, \vec{t}),\ \text{msg},\ (\vec{z}, c))$ — Simplified

1: Check that every coefficient of $\vec{z}$ is within the allowed bound (reject if not)
2: Recompute $\vec{w}' = A\vec{z} - c\vec{t}$
3: Accept if $c = H(\vec{w}', \text{msg})$; reject otherwise

Remark: Why Verification Works: the Algebra

Substitute: $\vec{w}' = A\vec{z} - c\vec{t} = A(\vec{y} + c\vec{s}_1) - c(A\vec{s}_1 + \vec{s}_2) = A\vec{y} + cA\vec{s}_1 - cA\vec{s}_1 - c\vec{s}_2 = A\vec{y} - c\vec{s}_2 = \vec{w} - c\vec{s}_2$. The two copies of $cA\vec{s}_1$ cancel — the same style of cancellation as Kyber's decryption in Lecture 3. $\vec{w}'$ isn't *exactly* $\vec{w}$; it's $\vec{w}$ shifted by a small quantity $c\vec{s}_2$. Real ML-DSA's "high bits" / hint mechanism (omitted above) is specifically engineered so that this small shift doesn't change the *high-order* bits of $\vec{w}$, so $H(\vec{w}', \text{msg})$ still reproduces the same challenge c the signer used. Our simplified version above sidesteps this by just checking exact equality of the full hash input in a toy example (Section 6) small enough that no shift occurs.

5 Why Rejection Sampling Exists (This Is the Important Part)

Remark: The Problem Rejection Sampling Solves

Look at line 4 above: $\vec{z} = \vec{y} + c\vec{s}_1$. If we *always* output $\vec{z}$, its distribution would depend — ever so slightly — on the secret $\vec{s}_1$, because $c\vec{s}_1$ is added on every single signature. Collect enough signatures on enough messages, and that slight dependence becomes a statistical attack surface: an attacker who sees many $(\vec{z}, c)$ pairs could, in principle, average out the masking $\vec{y}$ and recover information about $\vec{s}_1$. **Rejection sampling closes this gap**: $\vec{y}$ is deliberately sampled from a range wide enough that, *conditioned on being accepted*, $\vec{z}$'s distribution is statistically independent of $\vec{s}_1$ — indistinguishable from $\vec{z}$ having been sampled uniformly from the accepted range, with no trace of the secret at all. The rejected attempts simply never get published; only "safe-looking" signatures ever leave the signer.

Fact: Deterministic Signing, and Why It's a Double-Edged Sword

Step 1 above notes that $\vec{y}$ is derived *deterministically* from (sk, msg) via a pseudorandom function, rather than from fresh random bits each time. This mirrors a well-known fix (RFC 6979-style) for a classical signature failure mode: if two different messages were ever signed with the *same* $\vec{y}$ by accident (e.g. a broken random-number generator), the two resulting equations $\vec{z}_1 = \vec{y} + c_1\vec{s}_1$ and $\vec{z}_2 = \vec{y} + c_2\vec{s}_1$ can be subtracted to isolate $\vec{s}_1$ directly — a catastrophic, complete key-recovery, and a real failure class in classical ECDSA history (e.g. the Sony PlayStation 3 signing-key incident). Determinism eliminates the accidental version of this bug. But it introduces a new one: **if a fault attack can force the same $\vec{y}$ to be reused across two different signing operations** (by corrupting the PRF's internal state, or replaying an earlier internal value), the exact same subtraction attack becomes available again — this time caused deliberately rather than by accident. This is precisely the "fault attacks on determinism" line item flagged for Session 5: determinism is a defense against one failure mode and an attack surface for another, and real published Dilithium fault attacks exploit exactly this tension.

6 A Fully Worked Toy Example

Example: ML-DSA-Style Signing, $k = \ell = 1$, $n = 4$, $q = 17$ (Verified by Hand)

Reusing $R_{17} = \mathbb{Z}_{17}[x]/(x^4 + 1)$ and $A = 3 + 5x + 2x^2 + x^3$ from Lecture 3.
KeyGen. Let $s_1 = 1$ (constant polynomial) and $s_2 = 1 - x$ (small). Then

$$t = A \cdot s_1 + s_2 = A + (1 - x) = (3 + 1) + (5 - 1)x + 2x^2 + x^3 = 4 + 4x + 2x^2 + x^3.$$

Sign. Mask $y = x$. Commitment: $w = A \cdot y = A \cdot x = 16 + 3x + 5x^2 + 2x^3$ (computed exactly as in Lecture 2's ring-multiplication example). For this toy example, take the challenge to be the simplest possible sparse polynomial, $c = 1$ (constant). Response: $z = y + c \cdot s_1 = x + 1 = 1 + x$. (No rejection triggered — coefficients of z are small.) Signature: $(z, c) = (1 + x,\ 1)$.
Verify. Recompute $w' = A \cdot z - c \cdot t$:

$$\begin{aligned} A \cdot z &= A \cdot (1 + x) = A + A \cdot x = (3, 5, 2, 1) + (16, 3, 5, 2) \\ &= (19, 8, 7, 3) \equiv (2, 8, 7, 3) \bmod 17, \\ c \cdot t &= 1 \cdot t = (4, 4, 2, 1), \\ w' &= (2 - 4,\ 8 - 4,\ 7 - 2,\ 3 - 1) = (-2, 4, 5, 2) \equiv (15, 4, 5, 2) \bmod 17. \end{aligned}$$

Compare to the original $w = (16, 3, 5, 2)$: they are *not* identical — $w' = w - c \cdot s_2 = w - (1, -1, 0, 0)$, matching the algebra of Section 4's remark exactly (subtract $s_2 = 1 - x = (1, -1, 0, 0)$ from $w = (16, 3, 5, 2)$: $(16 - 1, 3 - (-1), 5, 2) = (15, 4, 5, 2) = w'$, confirmed). In this toy example we accept the signature by checking this exact algebraic identity holds (the real scheme's hint mechanism is what lets a verifier who does *not* know s_2 still confirm this identity, via the high-order bits of w instead of w itself — omitted here for tractability, as flagged in Section 4).

Exercise: Practice

Using the same A, s_1, s_2: sign the same message again, but this time with $y = 1$ (instead of $y = x$) and the same challenge $c = 1$. Compute the new z and confirm $w' = w - c \cdot s_2$ still holds with the new $w = A \cdot y$. Then, pretending you are an attacker who somehow learned that the *same* $y = x$ was used for two different messages with challenges $c_1 = 1$ and $c_2 = 2$ respectively, write down the two equations for z_1 and z_2 and show algebraically how subtracting them isolates s_1 — this is the calculation behind the factbox in Section 5.

7 FN-DSA (Falcon): a Different Lattice, a Different Technique

Falcon abandons MLWE entirely and uses a different hard problem on a different kind of lattice — worth seeing, precisely because your project needs to compare attack surfaces *across* schemes, not just understand one.

Definition: NTRU Lattices

Fix $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ as before. An NTRU key pair consists of two *short* polynomials $f, g \in R_q$ (the secret), from which the public key $h = g \cdot f^{-1} \bmod q$ is computed (assuming f is invertible mod q). The **NTRU lattice** associated to h is

$$\Lambda_h = \{\, (u, v) \in R_q^2 : u + v \cdot h \equiv 0 \pmod{q} \,\}.$$

(f, g) (suitably arranged) is a *short vector* in Λ_h — in fact, a short enough one to serve as a trapdoor *basis* for the whole lattice, not just a single short vector: this is the "all vectors short together" property Lecture 1's optional Minkowski's Second Theorem box was pointing toward.

Definition: The GPV Framework (Gentry–Peikert–Vaikuntanathan)

Given a trapdoor (short basis) for a lattice Λ, and a target point $t = H(\text{msg})$ (hashed into the same space as Λ), **sample** a lattice point $v \in \Lambda$ close to t, using the trapdoor to guide a randomized search — specifically, sampling from a discrete Gaussian distribution (Lecture 2, Section 4.5) centered near t. The signature is (essentially) the short offset $s = t - v$. Verification recomputes t from the message, checks s is short, and checks $s + v'$ (reconstructed from the public key and s) lands back on a valid target. Unlike Dilithium's commit-challenge-response shape, this is a single sampling step against a target — no interactive protocol being flattened, just a direct trapdoor-guided search.

Remark: Falcon's KeyGen/Sign/Verify, at a Glance

KeyGen: generate short f, g (this step itself is delicate number theory — specific polynomial sampling techniques — which we do not detail here); compute $h = gf^{-1} \bmod q$; the trapdoor is (f, g) (or an equivalent short basis derived from them), the public key is h. **Sign:** hash the message to a target point, then use the trapdoor with GPV sampling (Falcon's specific method is called *fast Fourier sampling*, chosen for speed) to find a short vector (s_1, s_2) with $s_1 + s_2 h \equiv H(\text{msg}) \pmod{q}$; output (s_1, s_2) (compressed) as the signature. **Verify:** recompute $s_1 = H(\text{msg}) - s_2 h \bmod q$ from the public h and the received s_2, and check that (s_1, s_2) is short enough to plausibly come from the trapdoor.

Looking Ahead: Why Falcon's Sampler Is the Star of Session 5

Two things make Falcon's signing step uniquely attack-prone compared to Dilithium's. First, GPV sampling needs to sample from a discrete Gaussian *exactly* (or very close to it) — an approximate or biased sampler can leak information about the trapdoor (f, g) through the statistical shape of many signatures, even without any single signature looking wrong. Second, Falcon's fast Fourier sampling is implemented using **floating-point arithmetic**, for speed — and floating-point operations have historically been far harder to make constant-time (free of secret-dependent timing or power variation) than the simple integer comparisons Dilithium's rejection sampling relies on. Put together: Dilithium's main defense (rejection sampling) is a young technique but algorithmically simple to implement safely; Falcon's main mechanism (Gaussian trapdoor sampling) is a mathematically older and better-understood technique, but a much harder *implementation* target to make side-channel-resistant. This is exactly the gap that produced the published single-trace and sampler-leakage attacks on Falcon that Session 5 will walk through.

8 Standardization Status

ML-DSA was finalized as **FIPS 204** alongside ML-KEM and SLH-DSA in August 2024, and is NIST's recommended general-purpose signature standard today.[1] FN-DSA (Falcon), by contrast, remains a **draft** standard (to become FIPS 206): as of mid-2026 it has not yet been finalized, with publication expected in late 2026 or early 2027, and major certificate authorities have stated they will not deploy it in production until it is.[2] Worth stating plainly in the proposal's related-work section: attacking a finalized federal standard (ML-DSA) and attacking a still-draft one (FN-DSA) carry different framing, even though both are technically live research targets today.

9 Comparing the Three Schemes So Far

	ML-KEM (Kyber)	**ML-DSA (Dilithium)**	**FN-DSA (Falcon)**
Hard problem	Module-LWE	Module-LWE	NTRU lattice (SIS/GPV-flavored)
Purpose	Key encapsulation	Signature	Signature
Core technique	MLWE encryption + FO transform	Fiat–Shamir-with-Aborts	GPV trapdoor sampling
Key defense mechanism	Implicit rejection (Decaps)	Rejection sampling	Exact Gaussian sampling
Riskiest implementation step	Re-encryption check	$\vec{y}$ generation / bound check	Floating-point sampler
Standard status (mid-2026)	FIPS 203, finalized	FIPS 204, finalized	FIPS 206, draft

Looking Ahead: Where This Leads: Session 4 and Session 5

You now have, for all three schemes, both the correct algorithm *and* a named "riskiest step." Session 4 will formalize the general distinction between attacking the math (everything in Lectures 1–4) versus attacking an implementation of it. Session 5 will then walk through real, published attacks on exactly the six risky steps identified across Lectures 3 and 4's summary tables — at that point, every attack should read as "targeting a specific line of an algorithm you can already write out from memory," not as an isolated trick.

[1] NIST, *Module-Lattice-Based Digital Signature Standard*, FIPS 204, August 2024.
[2] NIST Initial Public Draft, FIPS 206, submitted for public review in late 2025.

10 Exercises

Exercise: Homework Set

1. In your own words, explain the Fiat–Shamir transform to someone who has never seen an interactive proof — use the commit/challenge/response structure from Section 2, but invent your own non-cryptographic analogy (e.g. a game show, a magic trick) to illustrate why removing the interactive Verifier and replacing it with a hash still works.
2. Complete the practice exercise in Section 6 (two-signature key-recovery calculation) if you have not already.
3. Compare Kyber's re-encryption check (Lecture 3, Section 7) and Dilithium's rejection sampling (Section 5 above): both exist to prevent a secret-dependent quantity from becoming visible to an attacker. In 4–5 sentences, describe what each mechanism is protecting against, and why the two mechanisms look so different from each other despite that similarity of purpose.
4. Falcon's signature is a pair (s_1, s_2) with $s_1 + s_2 h \equiv H(\text{msg}) \pmod{q}$. Suppose $h = 5$ (a toy scalar stand-in, not a real ring element) and $H(\text{msg}) = 12 \pmod{17}$. If $s_2 = 2$, what must s_1 be for the equation to hold? Is this pair unique, or could other (s_1, s_2) pairs also satisfy the equation — and if so, what property (from Section 7) picks out the one Falcon actually outputs?
5. **Looking ahead:** using the comparison table in Section 9, predict — before Session 5 reveals the real answer — which of the three schemes' riskiest steps you would guess is *easiest* to defend against with constant-time coding practices alone, and which would need dedicated hardware countermeasures. Briefly justify your guess; we will revisit it directly in Session 5.